Pipeline de Observabilidad End-to-End con OpenTelemetry

Laboratorio 2 — Observabilidad en Ambientes Productivos

Maestría en Arquitectura de Software

Octubre 2026

Índice

1. Introducción **2**
1.1 Arquitectura del sistema 2

2. Modos de ejecución **3**

3. Decisiones de diseño **4**

4. Instrumentación con el SDK de OpenTelemetry **4**

5. El OTel Collector **5**

6. Backends y visualización **5**
6.1 Evidencias de ejecución 5

7. Análisis de overhead **7**

8. Infraestructura como código **8**

9. Conclusiones **8**

1. Introducción

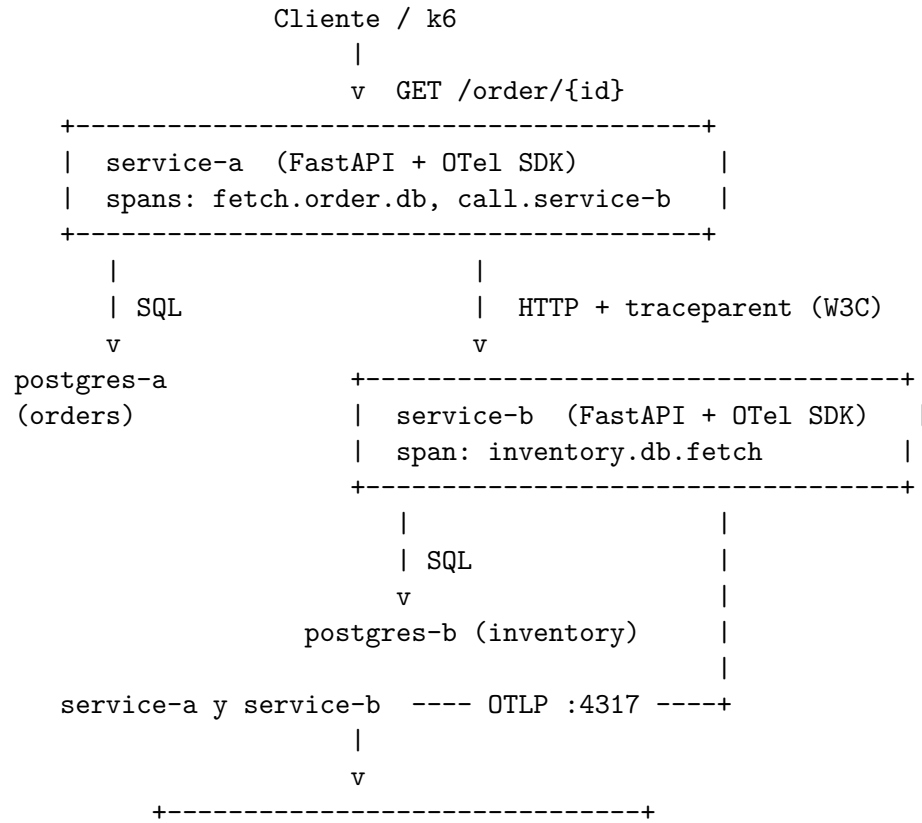
La observabilidad de sistemas distribuidos se apoya en tres tipos de señales complementarias: trazas, métricas y logs. Cuando estas señales se emiten de forma aislada resulta difícil reconstruir qué ocurrió durante un incidente. OpenTelemetry (OTel) propone un estándar común para instrumentar aplicaciones y transportar las tres señales de manera coherente, de modo que puedan correlacionarse entre sí.

El presente trabajo construye un pipeline de observabilidad completo sobre una arquitectura de dos microservicios con dependencia HTTP y acceso a base de datos. El objetivo es capturar las tres señales con el SDK de OpenTelemetry, procesarlas mediante un *OTel Collector* y visualizarlas en Jaeger, Prometheus/Grafana y Loki, logrando que un mismo `trace_id` permita navegar entre una métrica anómala, la traza que la originó y las líneas de log asociadas.

El sistema se compone de dos servicios escritos en Python con FastAPI. El primero, `service-a`, recibe las peticiones externas, consulta una base de datos de pedidos y delega en `service-b` la consulta de inventario. El segundo continúa la traza iniciada por el primero gracias a la propagación de contexto. Cada servicio administra su propia base de datos PostgreSQL, siguiendo el patrón de base de datos por servicio.

1.1 Arquitectura del sistema

La Figura 1 presenta la topología del pipeline. Una petición entra por `service-a`, que consulta su base de datos y delega en `service-b`; ambos emiten telemetría al Collector por OTLP y escriben logs a la salida estándar. El Collector distribuye cada señal al backend correspondiente, y Grafana permite correlacionarlas por `trace_id`.



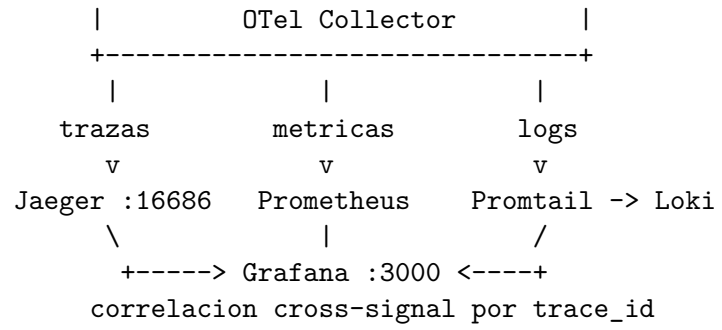


Figura 1. Arquitectura del pipeline de observabilidad end-to-end.

2. Modos de ejecución

El laboratorio se diseñó para operar en dos modos con configuración equivalente. El modo local, basado en Docker Compose, se usó para el desarrollo, las pruebas y la obtención de evidencias. El modo AWS, descrito mediante Terraform para ECS Fargate, constituye un entregable de infraestructura como código que se validó pero no se desplegó, con el fin de no incurrir en costos sobre una cuenta personal.

La siguiente tabla resume las diferencias entre ambos modos:

Aspecto	Modo local	Modo AWS
Orquestación	Docker Compose	ECS Fargate
Collector	Imagen contrib con bind-mount	Sidecar en la task definition
Backend de trazas	Jaeger	Jaeger sobre ECS
Transporte de logs	Promtail hacia Loki	Exporter awscloudwatchlogs
Estado de la infraestructura	Contenedores efímeros	terraform plan, sin apply

Mantener una única diferencia real entre los dos modos (el archivo de configuración del Collector y algunas variables de entorno) permite razonar sobre el comportamiento del sistema sin depender del entorno donde se ejecute.

3. Decisiones de diseño

Las decisiones de arquitectura más relevantes y su justificación se resumen a continuación.

Collector como sidecar. Se optó por desplegar el Collector junto a cada servicio en lugar de un gateway centralizado. Así los servicios emiten su telemetría a `localhost:4317`, se evita la necesidad de descubrimiento de servicios para el tráfico de telemetría y un fallo del Collector queda contenido en su propia unidad de despliegue.

Base de datos por servicio. Las tablas `orders` e `inventory` no comparten claves ni se consultan de forma conjunta, por lo que cada servicio recibió su propia instancia de PostgreSQL. Esta separación refleja un diseño de microservicios realista y refuerza la independencia entre componentes.

Métricas a través del Collector. El SDK envía las métricas por OTLP al Collector, que las expone en un endpoint que Prometheus consulta. Se eligió este flujo en lugar de exponer un endpoint de métricas por servicio, de modo que el punto de recolección sea único.

Transporte de logs mediante el driver de contenedores. Los servicios escriben logs en formato JSON a la salida estándar, incluyendo el `trace_id` y el `span_id` del contexto activo. En el modo local, Promtail recoge esa salida y la envía a Loki; en el modo AWS, el driver de CloudWatch cumple la misma función. Este patrón es el habitual en despliegues de contenedores.

Acceso restringido a las interfaces. Las interfaces de Jaeger, Grafana y Prometheus quedan expuestas, en el modo AWS, detrás de un grupo de seguridad que limita el acceso a la dirección IP del operador. Un balanceador público sin esta restricción quedaría expuesto a sondeos automatizados.

4. Instrumentación con el SDK de OpenTelemetry

Ambos servicios se instrumentaron con el SDK de OpenTelemetry para Python. La configuración incluye un `TracerProvider` con procesamiento por lotes y exportación por OTLP, y un `MeterProvider` con exportación periódica de métricas. Sobre esa base se habilitó la instrumentación automática de FastAPI, del cliente HTTP httpx y del controlador de base de datos psycopg2, que genera spans sin necesidad de modificar la lógica de negocio.

Adicionalmente se definieron spans propios para las operaciones de negocio relevantes. En `service-a` se instrumentaron la consulta a la base de datos de pedidos y la llamada al servicio de inventario; en `service-b`, la consulta de inventario y las operaciones sobre su caché. Los registros de log se emiten en formato JSON e incorporan el identificador de traza, que actúa como enlace entre las tres señales.

La propagación de contexto entre servicios merece una mención particular. En las primeras pruebas las trazas de ambos servicios aparecían separadas, sin un identificador común. El problema se originaba en la forma de activar la instrumentación de FastAPI: al no crearse el span de servidor como raíz, los spans propios quedaban desconectados. La solución consistió en instrumentar explícitamente la aplicación una vez construida, con lo cual una sola traza pasó a contener los spans de ambos servicios bajo el mismo `trace_id`.

5. El OTel Collector

El Collector se configuró para recibir telemetría por OTLP en sus variantes gRPC y HTTP. El procesamiento aplica, en orden, un limitador de memoria que protege al Collector bajo carga, el enriquecimiento con atributos de recurso, la detección de atributos del entorno, un filtro que excluye las verificaciones de salud del flujo de trazas y, por último, el agrupamiento por lotes previo a la exportación.

Las trazas se exportan hacia Jaeger mediante OTLP. Las métricas se exponen en un endpoint que Prometheus consulta, habilitando la conversión de atributos de recurso en etiquetas para poder filtrar por servicio. En el modo AWS, el flujo de logs se dirige a CloudWatch y la detección de recursos se ajusta al entorno de ECS.

6. Backends y visualización

Jaeger concentra la visualización de trazas distribuidas. Al consultar una traza se observa el árbol completo de la petición, con los spans de `service-a` y `service-b` agrupados bajo un mismo identificador, lo que confirma la propagación de contexto entre servicios.

Prometheus almacena las métricas que expone el Collector y Grafana las presenta en un tablero con indicadores de nivel de servicio: disponibilidad, latencia en distintos percentiles, tasa de error y rendimiento, además de paneles sobre el uso de CPU y el estado del propio Collector.

La correlación entre señales se materializa en Grafana. Desde una línea de log en Loki es posible saltar a la traza correspondiente en Jaeger usando el `trace_id`, y desde una traza es posible consultar los logs asociados. Esta navegación es la que habilita el diagnóstico de incidentes en tiempo real.

6.1 Evidencias de ejecución

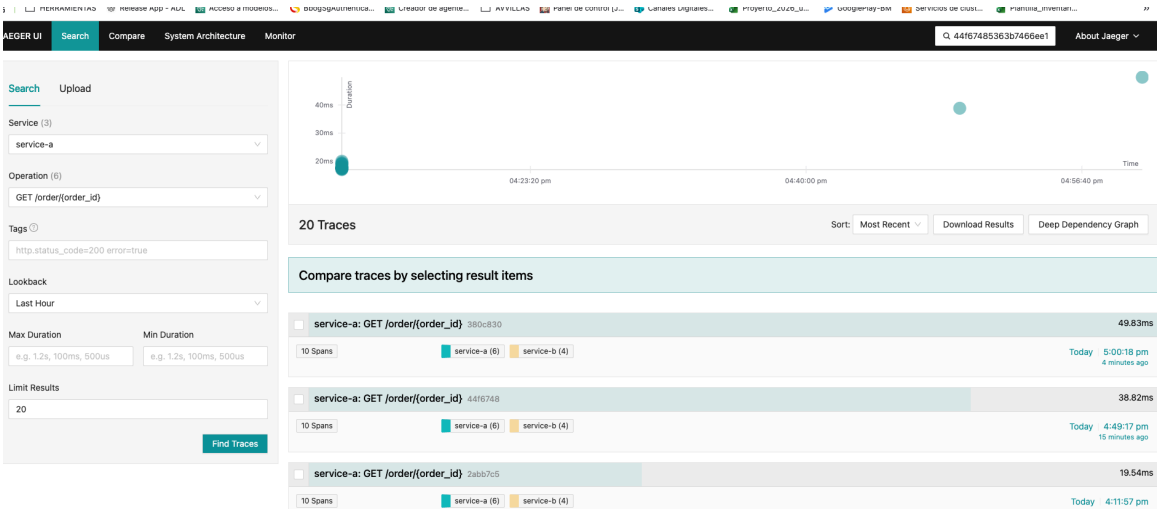


Figura 1: Figura 2. Jaeger UI. Listado de trazas de service-a; cada traza agrupa 10 spans que abarcan service-a y service-b, lo que evidencia la propagación de contexto entre servicios.

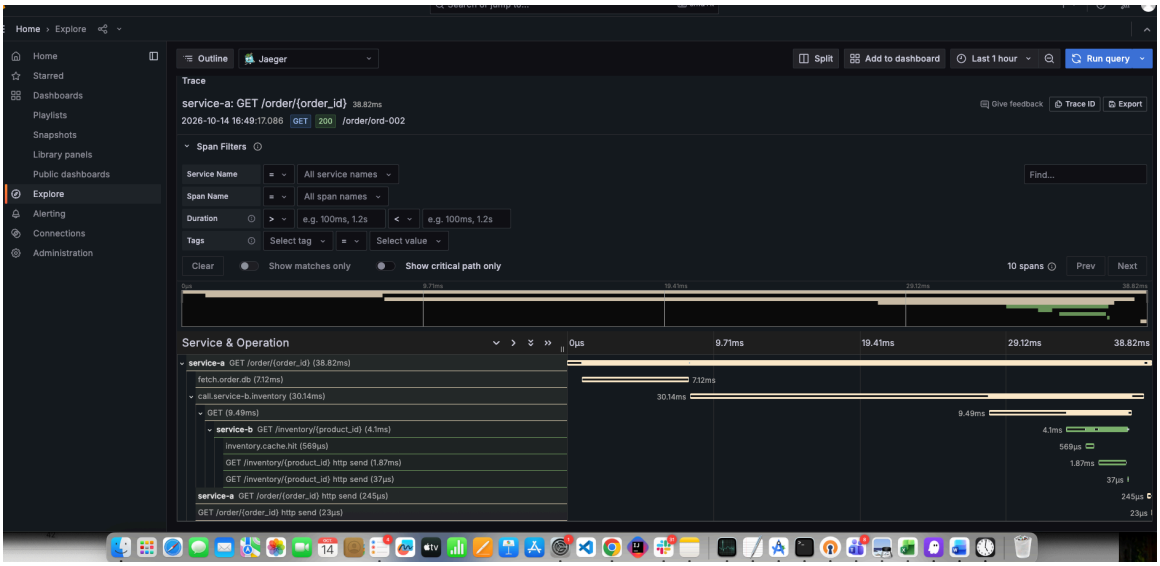


Figura 2: Figura 3. Grafana Explore con fuente Jaeger. Árbol de una traza distribuida: GET /order, fetch.order.db, call.service-b.inventory y los spans de service-b.

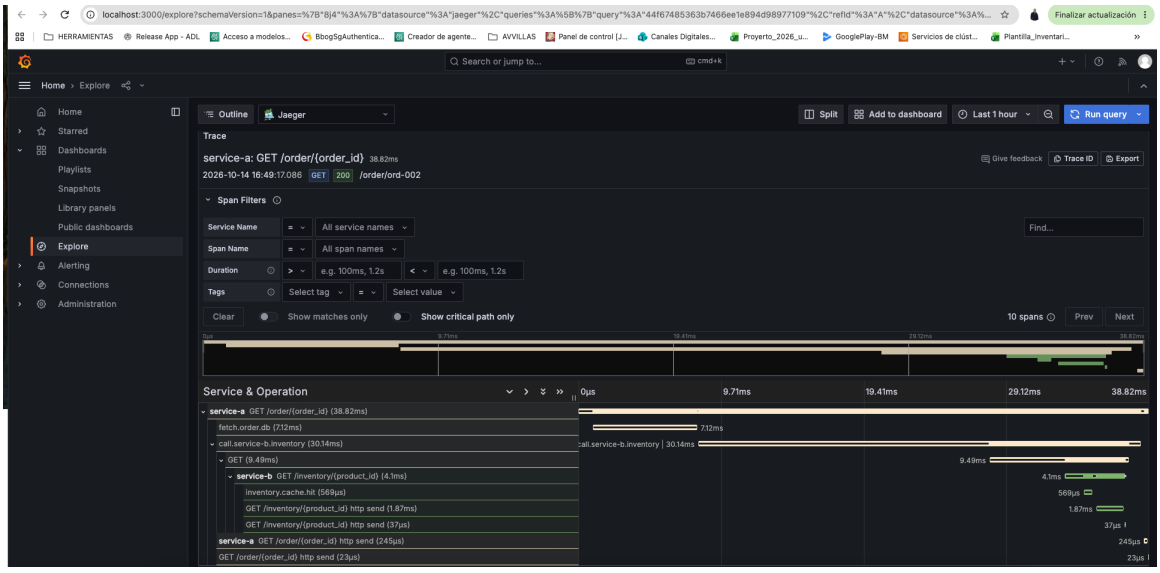


Figura 3: Figura 4. Grafana Explore. La misma traza abierta mediante búsqueda por trace_id, resultado del pivote de correlación.

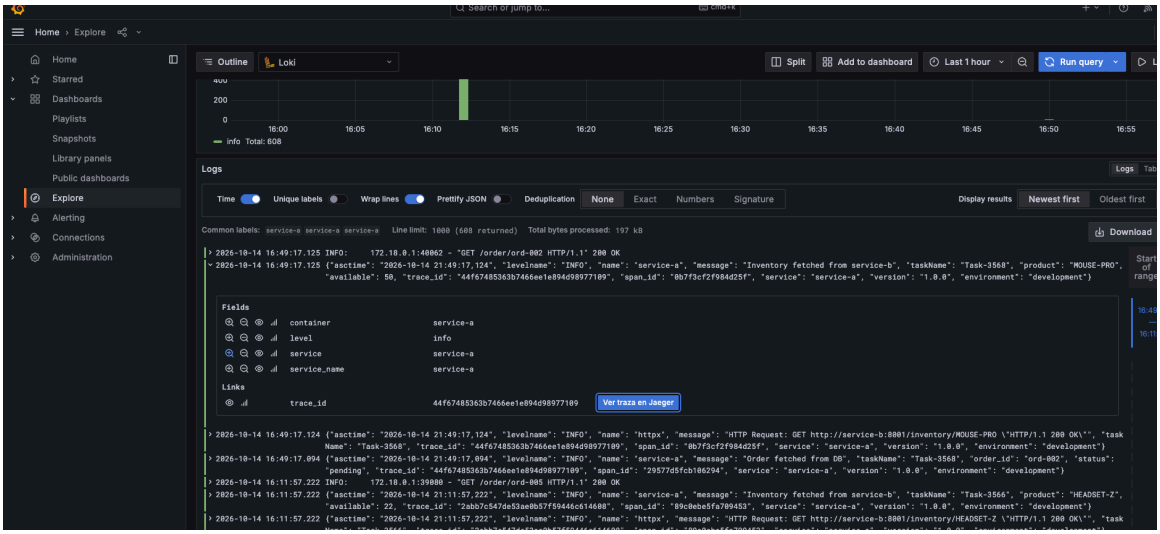


Figura 4: Figura 5. Grafana Explore con fuente Loki. Líneas de log en formato JSON con el trace_id y el enlace que permite saltar desde un log hasta su traza en Jaeger.

7. Análisis de overhead

Para medir el costo de la instrumentación se ejecutó una prueba de carga con k6 sobre dos variantes de la misma imagen de contenedor. La única diferencia entre ellas es una variable de entorno que activa o desactiva el SDK. Con el fin de aislar el costo de la instrumentación, se desactivó durante la medición la latencia de base de datos simulada, que de otro modo dominaría los tiempos de respuesta.

Métrica	Sin OTel	Con OTel	Variación
Latencia promedio (ms)	35.4	40.2	+13.6 %
Latencia p95 (ms)	81.5	88.4	+8.5 %
Latencia p99 (ms)	124.7	219.4	+76.0 %
Rendimiento (peticiones/s)	43.5	43.5	0.0 %
CPU, tres contenedores (%)	60.6	73.4	+21.3 %
Memoria, tres contenedores (MiB)	309.4	319.3	+3.2 %

Los resultados muestran un costo coherente con lo esperado para una instrumentación con agrupamiento por lotes activo. La latencia promedio y el percentil 95 crecen alrededor de un diez por ciento, el rendimiento se mantiene y el consumo de CPU aumenta cerca de un veintiuno por ciento. El percentil 99 presenta una variación mucho mayor, pero debe interpretarse con cautela: es una métrica de cola, sensible a eventos puntuales, medida sobre una máquina virtual de cuatro núcleos que comparte recursos con el propio Collector, por lo que su valor fluctúa de forma apreciable entre ejecuciones. Los indicadores estables son la latencia promedio, el percentil 95 y el consumo de CPU.

La prueba es reproducible: un script recrea los servicios con y sin instrumentación, toma muestras del consumo de recursos durante la carga y genera la tabla comparativa de forma automática.

8. Infraestructura como código

El despliegue en AWS se describió con Terraform usando la versión 5 del proveedor de AWS. La configuración se organiza en cinco módulos (red, registro de contenedores, roles de identidad, backends de observabilidad y aplicación) que suman alrededor de cuarenta recursos: una red virtual con subredes públicas y privadas, grupos de seguridad restringidos, repositorios de imágenes, roles con permisos acotados, un clúster ECS, el espacio de nombres para descubrimiento de servicios, las definiciones de tarea con el patrón de contenedor acompañante, los servicios, el balanceador de carga y la conexión entre servicios.

La infraestructura se validó mediante `terraform init`, `validate` y `plan`, confirmando que la descripción es correcta y que generaría los recursos previstos. No se ejecutó la aplicación de los cambios, en consonancia con la decisión de operar únicamente en local.

9. Conclusiones

Se construyó un pipeline de observabilidad funcional que captura y correlaciona las tres señales con OpenTelemetry. La correlación por `trace_id` entre logs, trazas y métricas permite navegar de un síntoma a su causa, que es el valor central de la observabilidad. El overhead medido resulta aceptable frente al beneficio que aporta la instrumentación.

La estrategia de desarrollar primero en local, con una configuración equivalente a la de AWS, permitió completar y evidenciar el laboratorio sin costos de nube, conservando una descripción de infraestructura válida y lista para desplegarse.

Como trabajo futuro se plantea realizar el despliegue efectivo en ECS, incorporar muestreo de trazas para reducir el volumen en producción y definir alertas sobre los indicadores de nivel de servicio con sus respectivos objetivos.